サイバー脅威インテリジェンスにおける LLM を用いた自己修復型クローリングシステムの提案と評価

野本 一輝[1,a)]　田中 優奈[1]　福永 拓海[1]　櫻井 悠次[1]　丸田 佳[1]　鈴木 芽依[1]　古川 凌也[1]
高田 雄太[1]　熊谷 裕志[1]　神薗 雅紀[1,2]

概要：サイバー脅威インテリジェンスでは，多数の Web サイトから継続的に脅威情報を収集する必要がある．しかし，XPath 等の固定ルールに基づくクローリングは，対象サイトのレイアウト変更によって記事を取得できなくなる．本稿では，平常時は静的なクローリングコードで低コストかつ高速にクローリングを実施し，クローリング失敗検出時は LLM で修正版コードを生成する，自己修復型クローリングシステムを提案する．本システムは，クローリング失敗の検出と分類，修復対象の選定，修正版コードの生成と試行実行による検証，およびアナリストの承認を経た設定への反映を，一連のワークフローとして実行する．検証用サイト群による評価では，失敗検出率と失敗分類精度がいずれも 1.000，正常系誤検出率が 0.000 であった．また，5 種類の LLM を用いた計 250 ケースのコード生成では，上位モデルが記事取得の再現率 1.000 を達成する一方，モデル間の性能差が大きいことを示した．さらに，3 段階の構造変更を与えた End-to-End 評価では，失敗検出から設定反映までの全段階が到達率 1.000 で動作し，人手によるコード修正を伴わずにクローリングが復旧した．一方，反映後の再現率は 0.844，適合率は 0.803 であり，実記事に類似する広告記事の除外と，クライアントサイドレンダリング環境における記事 URL の取得に課題が残った．

キーワード：サイバー脅威インテリジェンス，セキュリティ，クローリング，LLM

LLM-Based Self-Repairing Crawling System for Cyber Threat Intelligence

Kazuki Nomoto[1,a)]　Yuna Tanaka[1]　Takumi Fukunaga[1]　Yuji Sakurai[1]　Kei Maruta[1]
Mei Suzuki[1]　Ryoya Furukawa[1]　Yuta Takata[1]　Hiroshi Kumagai[1]　Masaki Kamizono[1,2]

Abstract: Cyber threat intelligence requires continuous collection of threat information from various websites. Crawling based on hand-crafted rules such as XPath expressions breaks when a target website changes its layout. In this paper, we propose a self-repairing crawling system. The system crawls with static code under normal operation, and it invokes an LLM to generate revised code only when consecutive crawling failures occur. A single workflow spans failure detection and classification, selection of repair targets, code generation and validation, and deployment after analyst approval. We evaluated the system on a controlled set of test websites. The failure detection rate and the failure classification accuracy were both 1.000, and the false-positive rate was 0.000. We also generated crawling code for 250 cases with five LLMs. The top models attained a recall of 1.000 in article retrieval, but performance varied widely across models. Finally, we changed the structure of each website three times and evaluated the whole system. Every stage completed in all cases, and crawling was restored without manual revision. The recall and precision after deployment were 0.844 and 0.803. The generated code did not exclude advertisement articles that resemble genuine ones, and it did not obtain article URLs under client-side rendering.

Keywords: Cyber Threat Intelligence, Security, Crawling, Large Language Model

1. はじめに

サイバー脅威インテリジェンス（Cyber Threat Intelli-

[1] デロイト トーマツ サイバー合同会社, Deloitte Tohmatsu Cyber LLC
[2] 一般社団法人デロイト トーマツ戦略研究所, Deloitte Tohmatsu Strategy Institute
[a)] kazuki.nomoto@tohmatsu.co.jp

© 2026 Information Processing Society of Japan

This paper is work in progress and not peer-reviewed.

gence: CTI）においては，セキュリティベンダの公開情報，ニュースサイト，専門フォーラム，アンダーグラウンドフォーラム等の多様な Web サイトを継続的に監視し，最新の情報を収集する基盤が必要である [1]．こうした情報収集では，対象サイトのレイアウト更新や DOM 構造，CSS セレクタ，ページネーション方式等の変更により，従来利用していたクローリング処理が突発的に動作しなくなる場合がある．クローリングの停止や情報の取得漏れは，脅威情報の把握およびインシデント対応の遅延につながり得るため，クローリングの失敗を早期に検知し，サイト構造の変化に対応できる仕組みが求められる．

上記の要件に対し，XPath 等の固定ルールに基づく方式は，対象ページの構造が一定である限り，比較的低い実行コストで安定的に動作する一方，構造変更時には人手による抽出ルールやコードの修正を要する [2]．また，LLM を用いて同一サイト内で再利用可能なスクレイパを生成する手法も提案されているが，継続運用中に生じるクローリング失敗の検出や，失敗を契機としたスクレイパの再生成は対象としていない [3]．これら既存研究では，抽出ルールの修復や LLM によるコード生成が個別に検討されているものの，クローリング失敗の検出からコード生成，検証，設定への反映までを一連のワークフローとして統合し，多数の Web サイトの継続監視を実現するシステムの設計やその手法については十分に議論されていない．

そこで本研究では，定常時は静的なクローリングコードを使用し，連続したクローリング失敗の検出時に限り，LLM による修正版コードを生成する，ハイブリッド型の自己修復クローリングシステムを提案する．提案システムは，クローリング失敗の検出・分類，修正版コードの生成，安全性検査，試行実行，取得結果の検証，およびクローリング設定への反映を一連のワークフローとして統合する．これにより，LLM の利用を必要な場合に限定しながら，サイト構造の変化に伴うクローリング失敗からの復旧を自動化する．

提案システムを評価した結果，クローリング失敗の検出・分類から修正版コードの生成・検証，設定への反映までの自己修復処理を実行できた．具体的には，失敗検出率および失敗分類精度はいずれも 1.000，正常系誤検出率は 0.000 であり，高い精度でクローリングの失敗を検知・分類できた．また，5 種類の LLM を用いた計 250 ケースのコード生成評価では，再現率 1.000 を達成した条件においても適合率は 0.909 から 0.948 となり，多くのケースで適切なクローリングコード生成を実現できる一方で，実記事と構造的に類似する decoy 記事の除外に課題が残った．また，3 段階のレイアウト変更を伴うクローリング対象のサイトにおいて，システムの End-to-End (E2E) 評価を実施した．その結果，失敗検出から修正版コードの生成・検証，クローリング設定への反映までの各段階が全ケースで動作し，人手によるコード修正なしにクローリングが復旧することを確認した．一方，反映後にサイトごとに算出した再現率・適合率の平均は 0.844・0.803 であり，クローリング対象の技術スタックによる差を確認した．

2. 背景

2.1 固定ルールに基づく Web クローリング

従来のルールベース Web データ抽出では，対象サイトごとにスクレイパを実装し，XPath 等の固定的な規則によって目的の要素を特定する方式が用いられてきた [2]．固定ルールは処理を再現しやすく，同一サイトの継続的な巡回に適している一方，ページ構造に依存するため，DOM 構造の変更によって抽出に失敗する場合がある．

2.2 抽出ルールの自動生成・修復

固定ルールの保守を自動化するため，抽出結果の異常を検知し，変更後のページから スクレイパを再生成する手法が提案されている．Lerman らは，過去の抽出データから学習した構造的特徴を用いて，スクレイパの動作を検証し，Web サイトの変更時にスクレイパを再生成する方式を提案した [4]．このような方式は，変更後にもクローリング対象の Web サイトにおいて，変更前の構造的特徴が残る場合には有効であるが，修復に利用できる手掛かりが大きく失われる変更への適用には制約がある．

2.3 LLM を活用したスクレイパ生成

近年では，LLM の HTML 理解能力とコード生成能力を利用して，再利用可能なスクレイパを生成する手法が研究されている．スクレイパを生成せず，ページごとに LLM の推論によって情報を抽出する方式は，未知の Web ページやサイト構造に柔軟に適用できる一方，各ページに対して LLM 推論を実行する必要があるため，大規模な継続収集では処理時間および計算・API 利用コストが累積する [5], [6]．AUTOSCRAPER は，HTML の階層構造と同一サイト内のページ間の類似性を利用してサイト単位で再利用可能なスクレイパを生成することで，抽出時の LLM への依存を低減する方式を提案している [3]．一方，同手法はスクレイパの生成および同一サイト内での再利用を主眼としており，継続運用中に生じる抽出失敗の検出や，失敗を契機としたスクレイパの再生成は扱っていない．

2.4 本研究の位置づけ

本研究では，通常時は事前に作成した固定のクローリングコードを用い，クローリング失敗検出時に限って LLM による修正版コードの生成を行うシステムを提案する．生成コードは，試行実行と抽出結果の検証を経てシステムに保存され，人手による承認後に適用する．したがって，本研究の主眼は新たな抽出・修復アルゴリズムの提案ではな

© 2026 Information Processing Society of Japan

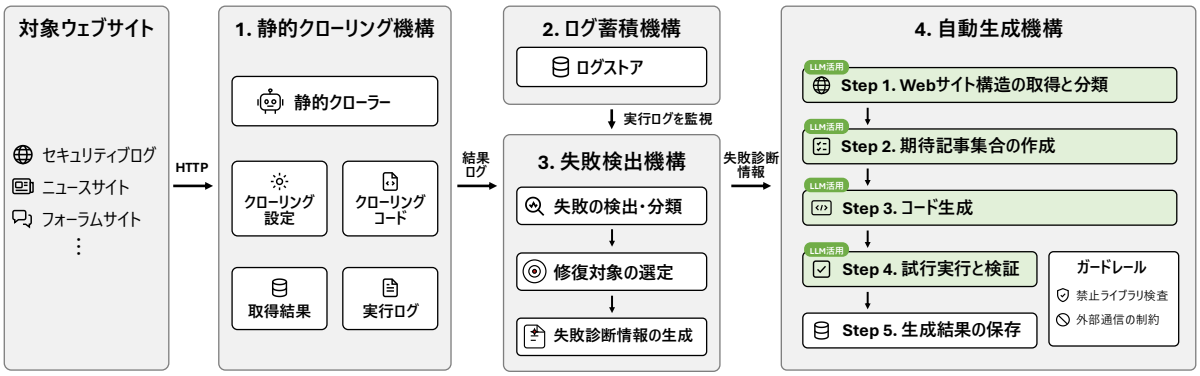


図 1 提案システムの全体像

く，固定コードによるクローリング，障害検出，LLM によるコード再生成，検証および承認を統合した一連の自己修復型クローリングシステムの設計と評価にある．

3. 提案手法

3.1 想定する運用環境と要件

本研究では，セキュリティベンダの公開情報，ニュースサイト，専門フォーラムなど多岐にわたる情報源を継続的に巡回し，最新の脅威情報や記事を定期的に収集する運用環境を想定する．このような多様な Web サイト群を対象とした長期運用では，Web サイトのレイアウト変更や DOM 構造の更新によって，既存のデータ取得が突発的かつ部分的に失敗する障害が頻発する．したがって，システム全体には多数の Web サイトを処理するための低コスト性および拡張性に加え，確実な障害検出可能性，および自律的な修復容易性が求められる．

本研究におけるクローリングタスクは，対象となる Web サイトのトップページ（またはインデックスページ）の URL が入力された際に，その Web サイトが発信する最新記事や更新情報を Web コンテンツとして適切に取得することと定義する．ここで，最新記事や更新情報が一覧として掲載されるページを一覧ページと呼ぶ．一覧ページは，Web サイトのトップページから特定され，以降の定常的なクローリングの起点として用いられる．一覧ページの特定は， 3.6 節の Step 1 において行われる．なお，取得した Web コンテンツから特定のデータ（IoC 等）を抽出する後段の解析処理は本研究のスコープ外とする．

3.2 システム全体像

提案システムは，静的クローリング機構，ログ蓄積機構，失敗検出機構，自動生成機構の 4 つのコンポーネントから構成される．提案システムの全体像を図 1 に示す．平常時は，静的なクローリングコードによって記事が取得され，データの取得結果と実行ログがログ蓄積機構に記録される．連続したクローリング失敗が検出されると，修復対象が自動的に選定され，失敗診断情報に基づいて修正版のクローリングコードが生成・検証される．生成結果は検証指標とともに draft として保存され，アナリストによるレビューと承認を経て既存のクローリング設定へ適用される．これにより，平常時の実行コストを抑えつつ，クローリング失敗時に自動的にクローリングが復旧されるクローリングシステムを実現する．

3.3 静的クローリング機構

静的クローリング機構では，監視対象の Web サイトごとに用意したクローリング設定に基づいてクローリングを実行し，記事データを記録する．クローリング設定は，一覧ページの URL，クローリングコード，取得件数の上限，および実行スケジュールから構成される．クローリングコードは Python により記述され，一覧ページの取得とページ送りの処理，および XPath や CSS セレクタ等による記事の抽出規則を内部に含む．定常巡回において決定論的な静的方式を採用することで，低コストかつ少ないリソースでクローリングを実行でき，並列分散による大規模化が可能となる．本機構では，クローリング時に，クローリングで用いられた各種設定，実行コード，取得結果，および動作ログを，ログ蓄積機構に送信する．これらの情報は，失敗原因の診断および修正版のクローリングコード生成時のコンテキストとして利用される．

3.4 ログ蓄積機構

ログ蓄積機構は，各クローリング実行結果をクローリング設定と対応付けて記録する．記録対象には，実行成否，HTTP ステータスコード，フェッチ回数，取得文書数，アクセス拒否の発生状況，実行時のエラーメッセージ，および動作ログが含まれる．また，同一のクローリング設定における連続失敗回数を算出し，診断情報として保存する．

蓄積されたログは，失敗種別の分類と修復対象の選定に用いられる．さらに，修正版のクローリングコードを生成する際には，直近の失敗診断情報と実行ログを LLM へ入力し，失敗原因を考慮したコード生成を実現する．

3.5 失敗検出機構

失敗検出機構では，各クローリング時に記録された診断情報から事前に定義されたルールに基づいて失敗を検知・分類する．失敗種別として，抽出結果なし（`zero_result`），フェッチ未実行（`zero_fetch`），HTTP エ

© 2026 Information Processing Society of Japan

ラー（http_error），接続エラー（connection_error），アクセス拒否（access_blocked），タイムアウト（timeout），スクリプトエラー（script_error），設定不備（config_error），およびその他の例外（exception）の 9 種類を定義する．失敗分類には，実行ステータス，HTTP ステータスコード，フェッチ回数，取得文書数，robots.txt およびドメインポリシーによるブロック件数，ならびに実行時のエラーメッセージを用いる．

本システムでは，各クローリング設定の診断情報を取得し，連続 3 回以上失敗しており，かつ失敗種別が修復可能と判定されたクローリング設定について，クローリングコード生成の対象として自動的に選定する．修復可能な失敗種別は，抽出結果なし，フェッチ未実行，スクリプトエラー，タイムアウト，およびその他の例外の 5 種類と定義する．HTTP エラー，接続エラー，アクセス拒否，および設定不備は，クローリングコードの書き換えでは解消しないため修復対象から除外する．なお，修復フローはスケジューラにより定期的に実行される．

3.6 自動生成機構

提案手法では，検出されたクローリングの失敗が修復対象として選定された場合，またはユーザから明示的な指示が与えられた場合に，LLM を用いて修正版の静的クローリングコードを生成する．なお，本機構を活用することで，新規の Web サイトに対するクローリングコードを生成することも可能である．本機構は，**5 つのステップ，Step 1. Web サイト構造の取得と分類，Step 2. 期待記事集合の作成，Step 3. コード生成，Step 4. 試行実行と検証，Step 5. 生成結果の保存**から構成される．

Step 1. Web サイト構造の取得と分類. まず，対象 URL から Web ページを HTTP アクセスにより取得する．静的 HTML の取得に失敗した場合，または記事一覧が JavaScript によって動的に生成されていると判断された場合には，ブラウザによるレンダリングへ切り替える．

次に，取得した Web ページ構造を LLM へ入力し，サイト種別，収集対象となるコンテンツカテゴリ，各カテゴリの一覧ページ，JavaScript レンダリングの要否の特定，およびペイウォールの有無を分類する．サイト種別には，ニュースサイト，ブログ，企業サイト，サービスサイト，政府・自治体サイト等を用いる．また，1 つのサイトにプレスリリース，お知らせ，ブログ等の複数の最新情報が存在する場合には，それぞれを異なるコンテンツカテゴリとして分類する．

最後に，一覧ページに含まれるページ番号，next リンク，offset，および cursor 等を分析し，ページネーション方式を判定する．ブラウザによるレンダリングの場合には，実行中に発生した通信も観測し，記事一覧を返す JSON API とそのカーソル構造を分析する．これにより，HTML 上に後続ページへのリンクが現れない動的な Web サイトにも対応する．ページネーションによる取得範囲は，処理コストを制限するため最大 5 ページとする．

Step 2. 期待記事集合の作成. Web サイトにおいて一覧ページを取得し，対象サイトが発信する記事に該当すると判断された URL とタイトルを期待記事集合として取得する．複数のカテゴリ，一覧ページ，および API レスポンスから得られた記事候補は，URL を正規化した上で重複を除去し，生成コードの内部検証に用いる期待記事集合として保持する．静的 HTML から得られる記事候補が著しく少ない場合には，JavaScript レンダリング後の Web ページから期待記事集合を再取得する．これにより，静的 HTML にはナビゲーション等の一部要素しか含まれず，記事一覧が動的に生成される Web サイトに対応する．なお，この期待記事集合は 後段の Step 4. 試行実行と検証 で用いられるデータであり，評価実験で使用する決定的な正解記事集合とは区別する．

Step 3. クローリングコードの生成. 本ステップでは，Step 1 および Step 2 で得られた情報を LLM へ入力し，対象サイトから記事を取得するクローリングコードを生成する．コード生成には，対象 URL，Step 1 で分類したサイト種別とコンテンツカテゴリ，取得した Web ページ構造，一覧ページ，およびページネーション情報を入力とする．既存コードを修復する場合には，これらに加えて，既存のクローリングコード，当該コードに対応するクローリング設定，失敗種別，フェッチ回数，取得文書数，および直近のエラーメッセージを入力とする．これにより，現在の Web サイト構造だけでなく，既存コードがどのように失敗したかを踏まえた修正版コードの生成が可能となる．

Step 4. 試行実行と検証. 生成コードは直ちに既存のクローリング設定へ適用せず，試行実行と検証を実施する．試行実行によって得られた記事の URL と期待記事集合を正規化して照合し，期待記事数，取得記事数，一致記事数，取得漏れとなった URL，想定外に取得した URL，および再現率を算出する．

生成コードに構文エラー，実行時例外，禁止された外部通信機能の利用，または再現率の不足が検出された場合には，エラー内容と照合結果を LLM へフィードバックする．LLM は，取得漏れや想定外 URL 等の差分を基にコードを修正し，再度試行実行と検証を行う．本実装では，この生成・実行・検証の処理を最大 5 回まで繰り返す．

Step 5. 生成結果の保存. 最終的に，生成コード，期待記事集合，取得結果，再現率，取得漏れおよび想定外 URL を保存する．

3.7 ガードレール

LLM によって自動生成されたコードを実環境に投入するためには，システムに対する安全性と妥当性を担保する

© 2026 Information Processing Society of Japan

「ガードレール」が不可欠である．提案手法では，生成コードに対して，管理された取得経路を迂回し得る代表的な通信・ブラウザ操作ライブラリの利用を静的に検査し，出力データ形式を制約する．これにより，意図しない外部通信や，不適切な修復によるシステム全体への影響を低減する．

なお本研究では，完全な自律実行を目的とせず，生成コードを静的に検査した上で，期待記事との照合結果および品質スコアとともに保存する．生成コードは，アナリストによるレビューと承認を経た場合にのみ既存のクローリング設定へ適用される．このように，生成時の制約，検証，および人手による最終確認を組み合わせることで，コード生成の柔軟性と安全性の両立を図る．

3.8 実装

提案システムは Python 3.12 で実装した．クローリング基盤は FastAPI，Celery，RabbitMQ，PostgreSQL で構成し，コンテンツ取得にはレンダリングを伴わないアクセスに aiohttp，動的レンダリングに Playwright，HTML 解析に Beautiful Soup を用いる．すべてのネットワークアクセスは共通の取得処理に集約し，アクセス前に robots.txt およびドメインポリシーを確認する．

4. 評価

4.1 実験設定

提案手法の有効性を検証するため，失敗の検出・失敗の分類性能，クローリングコードの自動生成性能，および提案システム全体の E2E 動作を評価する．本評価では，他手法との網羅的な比較は行わず，複数のページネーション方式や多様な技術スタックで構築された検証用サイト（以下，統制環境）における提案システム単体の性能評価に焦点を当てる．

失敗検出・分類性能については，失敗検出率，失敗分類精度，および正常系に対する誤検出率を評価指標とする．コードの自動生成性能については，各サイトにあらかじめ定めた正解記事集合と自動生成されたクローリングコードにより抽出された記事集合を照合し，再現率および適合率を算出する．また，複数の LLM を用いて同一条件でコード生成を行い，モデル間の性能差も比較する．なお，実サイトに対する長期運用時の評価や運用コストの定量評価は本評価の対象外とし，今後の課題とする．

4.2 評価対象

データセットの構成． 本評価には，統制環境で動作する In the lab データセットを用いる．In the lab データセットは，ページネーションやレイアウト変更を伴う Web サイトに対するコード生成および自己修復性能を評価するサイト群 A，多様な技術スタックに対するコード自動生成性能を評価するサイト群 B，およびクローリング失敗の検出・分類性能を評価するサイト群 E から構成される．

表 1 In the lab 検証用データセット

名称	サイト構造分類	データセット数	
		小計	合計
サイト群 A	静的 HTML	4 件	20 件
	WordPress	4 件	
	React	4 件	
	Next.js	4 件	
	API ベース	4 件	
サイト群 B	静的 HTML	3 件	30 件
	WordPress	3 件	
	React	3 件	
	Next.js	3 件	
	API ベース	3 件	
	Django	3 件	
	Rails	3 件	
	Vue	3 件	
	Nuxt	3 件	
	Angular	3 件	
サイト群 E	失敗・正常系の再現（6 カテゴリ ×3 設定）	18 件	18 件

表 2 サイト群 E の検証条件と再現方法

カテゴリ	再現方法	正解ラベル
正常系	正常な記事一覧と 30 件の記事を返す	None
抽出結果なし	クローリングコードのセレクタと一致しない要素を返す	zero_result
フェッチ未実行	ネットワークアクセス前に処理を失敗させる	zero_fetch
HTTP エラー	HTTP 503 を返す	http_error
接続エラー	接続を受け付けないポートへアクセスさせる	connection_error
アクセス拒否	robots.txt によって対象パスへのアクセスを拒否する	access_blocked

サイト群 A および B の共通構成． サイト群 A および B の技術スタック別の構成を表 1 に示す．サイト群 A および B の各サイトは，正解記事 30 件と decoy 記事 3 件，およびそれら 33 件へのリンクを掲載する記事一覧ページから構成される．decoy 記事は，記事一覧に通常記事と同様の形式で表示されるインフィード広告を模した記事であり，正解記事と同一の要素構造・クラス名を持つが，正解記事集合には含めない．なお，3 件の decoy 記事には，それぞれ `Sponsored`・`PR`・`Promoted` のラベルをリンクタイトルの接頭辞として与える．また，いずれの decoy 記事についても，記事リンクに `rel="sponsored nofollow"` 属性を付与する．記事一覧と同一ページ内のナビゲーション領域には，`/about`・`/tags/cve`・`/authors/seclab`・`/rss.xml` のように記事詳細ページを持たないリンク（以下，ノイズリンク）を配置する．ノイズリンクは，記事リンクを選択するセレクタの指定範囲を過度に広く取った場合に記事として誤抽出されることを想定しており，記事一覧に並ぶ decoy 記事とは異なる性質の誤抽出源である．

サイト群 A． サイト群 A は，5 種類の技術スタックを用いて各 4 サイト，計 20 サイトで構成される．各サイトは，管理用 API を用いることで v1 から v4 までの 4 種類のレイアウト構造に切り替えられる（表 3）．記事一覧の提供方式は技術スタックにより異なり，静的 HTML・WordPress・Next.js では HTML で，API スタックでは HTML と JSON の両形式で，React では同一オリジンの JSON API で提供される．バージョンごとに，HTML で

© 2026 Information Processing Society of Japan

は記事カードの要素構造とクラス名が，JSON では記事配列のキー名と各記事のフィールド名が変化する．

ノイズリンクは，v1 と v2 では配置せず，v3 では `/about` や `/rss.xml` のようなナビゲーション由来のリンク 10 件，v4 ではこれに `/intel/category/ransomware` のように記事 URL と同じ名前空間をもつリンクを加えた 22 件とする．v4 では記事 URL 自体も `/intel/{id}` へ変更されるため，URL の接頭辞のみでは記事リンクを判別できない．また v4 では，`onclick` 属性のみを与えた `span` 要素として，遷移先を持たないリンク風要素を 2 件配置する．なお，コード自動生成性能の評価では，全 20 サイトを v4 へ切り替えた上で検証を実施する．

サイト群 B. サイト群 B は，10 種類の技術スタックを用いて各 3 サイト，計 30 サイトで構成される．各サイトは，正解記事 30 件と decoy 記事 3 件を単一ページに掲載する一覧に，サイト群 A の v3 と同じノイズリンク 10 件を加えた構成とし，ページ送りとレイアウトの切り替えは行わない．

サイト群 E. サイト群 E の検証条件とその再現方法を表 2 に示す．サイト群 E は，5 種類の失敗と 1 種類の正常系の計 6 種類について，それぞれ 3 つのクローリング設定を用意した計 18 設定で構成される．なお，本評価では，3.5 節で定義した 9 種類の失敗種別のうち，表 2 に記載した主要な 5 種類を対象とする．サイト群 E の再現方法は 2 種類に大別される．正常系，抽出結果なし，HTTP エラー，およびアクセス拒否の 4 種類は，検証用サイト側の応答によって再現する．これらは 1 設定につき 1 サイトを対応させるため，計 12 サイトを用意する．一方，フェッチ未実行と接続エラーの 2 種類は，クローリング設定側で再現する．フェッチ未実行にはネットワークアクセス前に処理を失敗させるクローリングコードを与え，接続エラーには接続を受け付けないポートを一覧ページ URL として与える．したがって，これら 6 設定に対応する検証用サイトは存在しない．

サイト群 E では，記事一覧は正解記事 30 件のみとし，decoy 記事とノイズリンクは配置しない．また，クローリングコードは 2 種類のみを用いる．フェッチ未実行の 3 設定には，前述のとおりネットワークアクセスの前に処理を失敗させるコードを与え，残る 15 設定には，一覧ページを取得し，固定のセレクタで記事リンクを抽出したうえで各記事ページを取得するコードを与える．これにより，失敗種別の差異が，クローリングコードではなくサイトの応答および一覧ページ URL の指定のみに起因する形での検証を実現する．

クローリング設定. 検証用サイトに対するクローリング設定は，サイト群 A および B では 1 サイトにつき 1 つを用意し，取得件数の上限 (`max_articles`) は，サイト群 E も含めて全設定で 45 件に統一する．

表 3 サイト群 A の各レイアウトバージョンの仕様

版	記事リンクの掲載	ページ送り	記事 URL	ノイズリンク
v1	33 件（1 ページ）	なし	`/articles/{id}`	0 件
v2	33 件（1 ページ）	なし	`/articles/{id}`	0 件
v3	11 件 ×3 ページ	`?page=`	`/articles/{id}`	10 件
v4	11 件 ×3 ページ	`?cursor=`	`/intel/{id}`	22 件

表 4 サイト群 E における失敗検出・分類評価結果

評価指標	分子／分母	評価値
失敗検出率	15/15	1.000
失敗分類精度	15/15	1.000
正常系誤検出率	0/3	0.000

4.3 失敗検出・分類性能の評価

提案手法の失敗検出機構が，クローリングの失敗を検知し，その要因を正しく分類できるかを評価する．本評価には，失敗条件と正解ラベルをあらかじめ固定したサイト群 E を用いる．

評価方法. サイト群 E に含まれる 5 種類の異常系と 1 種類の正常系の計 18 設定について，クローリングを 1 回ずつ実行する．実行後に生成された診断情報から失敗種別を取得し，表 2 に示した正解ラベルと照合することで，失敗分類結果の正当性を検証する．

失敗検出率は，異常系 15 件のうち何らかの失敗として検出された割合とする．失敗分類精度は，異常系 15 件のうち診断結果が失敗種別の正解ラベルと一致した割合とする．正常系誤検出率は，正常系 3 件のうち失敗として判定された割合とする．

評価結果. 全 18 件の実行について診断情報が記録され，失敗の検出およびその分類が適切に行われることを確認した．結果を表 4 に示す．異常系 15 件はすべて失敗として検出され，失敗検出率は 1.000 であった．また，検出された失敗種別は全件で正解ラベルと一致し，失敗分類精度は 1.000 であった．正常系 3 件はいずれも正常と判定され，正常系誤検出率は 0.000 であった．

4.4 クローリングコードの自動生成性能の評価

新規に監視対象 Web サイトを追加する際に，自動生成機構が対象記事を取得可能なクローリングコードを生成できるかを検証する．本評価には，サイト群 A の全 20 サイトとサイト群 B の全 30 サイトを用いる．

評価方法. サイト群 A v4 およびサイト群 B の各サイトについて，自動生成機構を用いてクローリングコードを生成する．LLM として，Claude Sonnet 4.6，Claude Opus 4.5，Claude Haiku 4.5，GPT-5.2，および GPT-4o mini の 5 モデルを用いる．各モデルについて，各サイトで 1 回ずつ生成を行う．ただし，LLM プロバイダ側の一時的な障害による失敗に限り，最大 3 回まで再試行する．

評価には，各サイトが内部的に保持する記事 ID を正解データとして用いる．サイト群 A および B ともに記事 ID

© 2026 Information Processing Society of Japan

表 5　クローリングコードの自動生成機構の評価結果

モデル	サイト群 A（20 サイト）		サイト群 B（30 サイト）	
	再現率	適合率	再現率	適合率
Claude Sonnet 4.6	0.900	0.836	1.000	0.948
Claude Opus 4.5	1.000	0.909	1.000	0.948
Claude Haiku 4.5	0.650	0.658	0.967	0.891
GPT-5.2	1.000	0.909	1.000	0.918
GPT-4o mini	0.750	0.585	0.649	0.595

1 から 30 までの 30 件 を正解記事集合とし，記事 ID 31 から 33 までの 3 件の decoy 記事は正解集合に含めない．

各サイトについて，生成されたクローリングコードにより取得された記事の URL と正解記事の URL を照合する．照合結果から，以下の 2 指標を算出する．

- 再現率：正解記事 30 件のうち，取得された記事に含まれていたものの割合
- 適合率：取得された記事のうち，正解記事に該当したものの割合

モデルごとの評価値には，各サイトで算出した再現率および適合率のマクロ平均を用いる．

評価結果． Claude Opus 4.5 と GPT-5.2 は両サイト群で再現率 1.000 を達成し，新規サイトに対する自動生成が十分に機能することを確認した．また，クローリングコード生成に用いる LLM モデルによって，生成されるコードの性能が異なることが明らかとなった．

モデル別の再現率および適合率を表 5 に示す．再現率 1.000 を達成した Claude Opus 4.5 と GPT-5.2 においても，サイト群 A の適合率は 0.909 にとどまった．両モデルとも 20 サイト中 18 サイト以上で decoy 記事 3 件をすべて抽出しており，rel 属性やラベル文字列を手掛かりとした decoy 記事の除外に課題があることが確認された．また，記事を 1 件も正しく取得できず，再現率・適合率がともに 0.000 となったケースが 23 件生じた．いずれも Claude Sonnet 4.6（2 件），Claude Haiku 4.5（6 件），GPT-4o mini（15 件）で発生しており，Claude Opus 4.5 と GPT-5.2 では生じていない．うち 14 件は，記事一覧の抽出には成功していたものの，各記事の URL を実在しない形式で構成していた．この失敗は，生成コードが一覧ページ上のリンクをたどるのではなく，API 応答に含まれる識別子から URL を組み立てていたことに起因する．クライアントサイドレンダリングを採用するサイトでは，組み立てた URL が実在しない場合でもフォールバックにより HTTP 200 を応答するため，生成コードの試行実行では誤りが表面化しない．以上の結果から，生成コードに URL の構成規則を推測させず，描画後の DOM から抽出したリンクを用いるようプロンプトで制約するとともに，検証時には取得結果の件数だけでなく URL の実在性を確認する必要があることが示唆された．

4.5　提案システムの E2E 評価

本節では，提案システムについて，失敗検出，修復対象の選定，修正版コードの生成・検証，クローリング設定への反映，および正常取得の再開までの一連の処理を完遂できるかを評価する．

評価方法． 評価では，サイト群 A の全 20 サイトを用い，LLM には Claude Sonnet 4.6 を用いる．各サイトを v1 の状態に設定し，事前に用意した静的クローリングコードによって正解記事を取得できることを確認した状態を初期状態とする．その後，各サイトの構造を v1 から v2，v2 から v3，v3 から v4 へ順次変更し，各変更について失敗の検出およびクローリングコードの自動生成処理を実行する．したがって，評価対象は 20 サイト × 3 回の構造変更からなる計 60 ケースとなる．

各構造変更後にクローリングを 3 回実行し，連続失敗として診断情報が記録されることを確認する．連続失敗が規定回数に達すると，ワークフローが修復対象を選定し，最新の Web ページ構造，既存のクローリングコード，および失敗診断情報を用いて修正版コードを生成する．生成コードは，試行実行と期待記事との照合によって検証され，その再現率および品質スコアとともに draft として保存される．

本番運用では draft の適用前にアナリストによるレビューと承認を必要とするが，本評価では E2E での動作を検証するため，生成された draft を自動的に適用する．適用後に再度クローリングを実行し，構造変更後の正解記事集合に対する再現率および適合率を算出する．

評価指標． 各ケースについて，以下の処理が完了したかを記録する．

(1) 連続失敗の検出
(2) 修正版コードの生成および検証
(3) 生成コードのクローリング設定への反映
(4) 反映後のクローリングの完走

本評価では，各段階の到達率と，反映後の再現率・適合率のマクロ平均を主要な評価指標とする．再現率および適合率は，4.4 節と同一の定義を用い，構造変更後の正解記事集合 30 件を分母とする．

評価結果． 全 60 ケースにおいて，失敗検出から正常取得の再開までの一連の処理が動作することを確認した．一方，再現率は 0.844，適合率は 0.803 にとどまり，対象記事を過不足なく取得する点には課題が残ることが明らかとなった．表 6 に，構造変更段階ごとの E2E 評価結果を示す．

技術スタック別の再現率／適合率に着目すると，静的 HTML が 1.000 ／ 0.955，WordPress が 1.000 ／ 0.947，Next.js が 0.944 ／ 0.920，API ベースが 0.778 ／ 0.737，React が 0.500 ／ 0.455 であった．サーバ側で記事一覧が出力される静的 HTML・WordPress・Next.js の 3 スタックが再現率 0.944 以上となった一方，API ベースおよびク

© 2026 Information Processing Society of Japan

表 6 提案システムの E2E 評価結果

構造変更	ケース数	失敗検出	生成	適用	検証完走	再現率	適合率
v1 → v2	20	20/20	20/20	20/20	20/20	0.800	0.782
v2 → v3	20	20/20	20/20	20/20	20/20	0.833	0.795
v3 → v4	20	20/20	20/20	20/20	20/20	0.900	0.832
合計	**60**	**60/60**	**60/60**	**60/60**	**60/60**	**0.844**	**0.803**

表 7 E2E 評価における修正版コード適用後の取得結果の分類

主な要因（該当スタック）	件数	再現率	適合率	抽出件数	重複除去後	decoy 抽出数
decoy 記事を区別せず抽出（複数スタック）	33	1.000	0.909	33	33	3/3
decoy 記事を正しく除外（複数スタック）	15	1.000	1.000	30	30	0/3
記事 URL の構成誤り（React）	6	0.000	0.000	33	33	—
1 ページ目のみを繰り返し取得し上限に到達（API ベース）	3	0.333	0.222	45	11	1/3
ページネーション未対応で 1 ページ目のみ取得（Next.js）	2	0.333	0.909	11	11	1/3
同一ページの二重取得により 11 件が重複（Next.js）	1	1.000	0.682	44	33	3/3

ライアントサイドで描画される React では再現率が大きく低下していた．

クローリングコードによる記事取得結果の内訳． クローリングコードによる記事取得の誤りは，正解記事の取得には成功しつつ不要な記事を除外できない類型（33 ケース）と，記事 URL の構成の誤り，ページ遷移の誤り，誤った重複抽出により取得結果が破綻する類型（12 ケース）に大別される．60 ケースを反映後の再現率・適合率・抽出件数の組み合わせで分類した内訳を表 7 に示す．

最多の要因は decoy 記事の混入であり，33 ケースで正解記事 30 件をすべて取得した一方，decoy 記事 3 件を区別せず抽出した．decoy 記事を正しく除外し，再現率・適合率がともに 1.000 となったのは 15 ケースであった．

次に多い要因は，React により構築された Web サイトにおける記事 URL の構成誤りであり，6 ケースで生じた．これは 4.4 節で述べた，各記事の URL を実在しない形式で構成する現象と同一である．なお，これらのケースでは抽出された URL がいずれも実在しないため，decoy 記事の抽出数は判定できない．

残る 6 ケースは，いずれもページ送り処理の失敗によるものであった．内訳は，1 ページ目のみを繰り返し取得して上限に達したもの（3 ケース），1 ページ目を取得した後に後続ページへ進まなかったもの（2 ケース），および全ページを取得したうえで同一ページを二重に取得したもの（1 ケース）であった．このうち 1 ページ目のみを取得した 5 ケースで decoy 記事の抽出が 1 件にとどまっているのは，decoy 記事の除外に成功したためではなく，2 ページ目以降に配置された残り 2 件へ到達していないためである．

5. 制約事項

本研究における 6 つの制限事項を述べる．第一に，失敗検出は取得ページ数や HTTP ステータスコードに基づくため，部分的な取得漏れや誤った記事の取得などの，取得ページ数が 0 件とならない失敗は検出できない．第二に，本稿の評価は著者らが構築した統制環境上の検証用サイトを対象としており，実際の Web サイトにおける多様な構造変更や運用条件に対する有効性は評価できていない．また，各条件について 1 回のみ検証しており，LLM の生成結果のばらつきや長期運用時の可用性についても評価していない．第三に，本稿では，定常巡回には静的なクローリングコードを用い，必要時にのみ LLM を利用することで運用コストの低減を図る手法を提案しているが，LLM の呼び出し回数，トークン消費量，処理時間などの運用コストについて定量的に評価していない．第四に，自動生成コードに対して導入したガードレールについて，生成コードの安全性や運用上の有効性を定量的に評価していない．第五に，評価対象は Surface web に限られ，Deep/Dark web 上のフォーラム等への適用検証が残されている．第六に，LLM を用いてスクレイパを自動生成する既存手法 [3] との定量的な比較は行っていない．

6. まとめ

本研究では，平常時は静的なクローリングコードで記事を取得し，クローリング失敗の検出時に限って LLM によるコード自動生成で修復する，ハイブリッド型の自己修復クローリングシステムを提案した．統制環境での評価により，失敗検出率と失敗分類精度がいずれも 1.000 であること，および失敗の検出・分類から修正版コードの生成・検証，設定への反映までが人手によるコード修正を伴わずに動作することを示した．一方で，反映後の記事集合には decoy 記事の混入や記事 URL の構成誤りが残り，再現率は 0.844 にとどまることも明らかとなった．今後は，長期運用時の可用性の評価，既存手法との定量的な比較，ガードレール設計の評価，および Deep/Dark web などレイアウト構造が複雑な Web サイトへの適用検証を進める．

参考文献

[1] Paris Koloveas, et al. intime: A machine learning-based framework for gathering and leveraging web data to cyber-threat intelligence. *Electronics*, Vol. 10, No. 7, 2021.

[2] Emilio Ferrara, et al. Web data extraction, applications and techniques: A survey. *Knowledge-Based Systems*, Vol. 70, pp. 301–323, 2014.

[3] Wenhao Huang, et al. AutoScraper: A progressive understanding web agent for web scraper generation. In *Proceedings of the 2024 Conference on Empirical Methods in Natural Language Processing*, 2024.

[4] Kristina Lerman, et al. Wrapper maintenance: A machine learning approach. *Journal of Artificial Intelligence Research*, Vol. 18, p. 149–181, February 2003.

[5] Pablo Landeta-López, et al. LLMs applied to web scraping and web crawling: a systematic review. *Computing*, Vol. 108, No. 6, 2026. Article 78.

[6] Anna Hätty, et al. A cost-efficient modular sieve for extracting product information from company websites. In Franck Dernoncourt, et al., editors, *Proceedings of the 2024 Conference on Empirical Methods in Natural Language Processing: Industry Track*, pp. 1444–1456. Association for Computational Linguistics, November 2024.

© 2026 Information Processing Society of Japan

研究倫理

本研究は，サイバー脅威インテリジェンスに資する情報の継続的な収集を目的としており，対象サイトに対する不正アクセスや著作権の侵害を意図するものではない．本稿の評価は，著者らが構築した統制環境上の検証用サイトを対象として実施した．実験中はシステムの動作を監視し，意図しない外部サイトへのアクセスが生じていないことを確認した．

また，自動生成されたクローリングコードがネットワークへアクセスする経路を，あらかじめ用意したインタフェースに限定した．当該インタフェースには，robots.txt に基づくアクセス可否の判定，クロール間隔の制御，および事前に登録したブラックリストに基づくリクエストの遮断が組み込まれている．さらに，生成コードがこのインタフェースを迂回する通信手段を用いていないことを，実行前の静的検査によって確認する．

今後の実運用では，生成されたコードをアナリストがレビューし承認したうえで反映する Human-in-the-loop の構成を前提として，脅威情報の収集を実施する予定である．

© 2026 Information Processing Society of Japan